

CS 551 Systems Programming

Lecture 4: Memory, Pointers, and Box

Why this lecture exists

The whole course in one sentence

The whole course in one sentence

Rust lets you write code as low-level as C, *and* the compiler refuses to let you ship the bug classes that have eaten the systems-programming world for fifty years.

The mechanism is the **borrow checker**.

To make sense of the borrow checker, you need a working mental model of:

1. Where a value lives (stack vs heap).
2. What a pointer actually is (and what a *fat* pointer is).
3. How ownership decides who is allowed to read and write a value.

Today's lecture is those three things.

By end of this lecture, you can...

1. Draw a memory diagram showing the stack and the heap, and place a `Box<T>` correctly in it.
2. Explain why `&[T]` is *two machine words* and `&i32` is one.
3. Walk through the cons-list example from *The Rust Programming Language* and say why it needs `Box`.
4. Predict, before compiling, whether a borrow checker will accept or reject a small snippet.
5. Pick between `Box<T>` and `Rc<T>` based on how many owners a value needs.

Today's path

1. **Stack and heap.** Where values live.
2. **References and fat pointers.** What a pointer carries.
3. `Box<T>`. Your first smart pointer.
4. **The list.** The canonical example, two ways.
5. **The borrow checker.** Ownership, moves, borrows.
6. `Rc<T>`. When one owner isn't enough.

Two supplements bracket this lecture. If "stack," "heap," and "pointer" are only vaguely familiar, read `stack-and-heap-primer.md` first. If you have written C or C++, read `coming-from-c.md` instead: it maps today's model onto what you already know.

1. The stack and the heap

Where does a value live?

Two places a value can live

Every value your program creates lives in one of two regions of memory:

- **The stack.** A region the function call mechanism manages for you. Cheap, ordered, automatic.
- **The heap.** A region you ask for explicitly when the value's size or lifetime doesn't fit the stack's rules.

Both are just bytes in RAM. The difference is *who decides when those bytes are reused*.

The stack

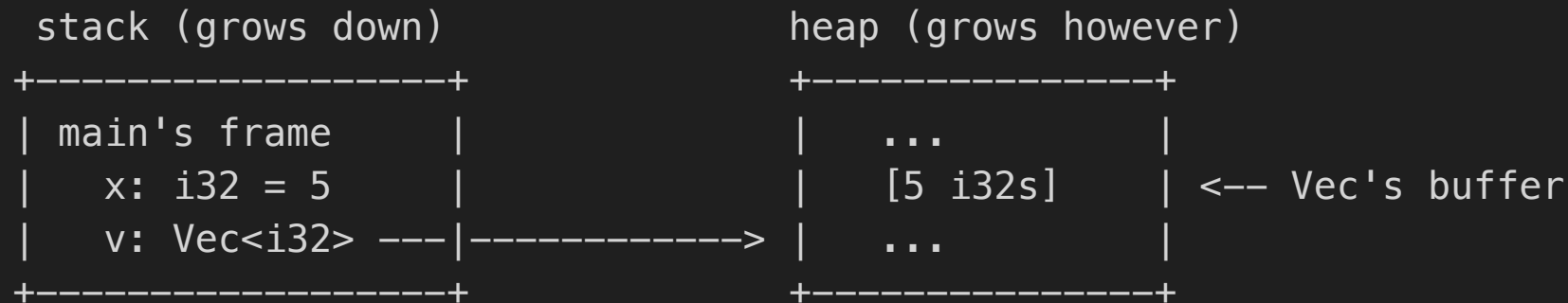
- **Layout.** A literal stack of *frames*, one per active function call.
- **A frame** holds the function's local variables, with sizes known at compile time.
- **Allocation.** When a function is called, the frame is pushed. When it returns, the frame is popped. Both happen in a handful of instructions.
- **Constraint.** Every value on the stack must have a size the compiler knows. No growing arrays. No "depends on user input."

```
1 fn add(a: i32, b: i32) -> i32 {  
2     let sum = a + b;    // a, b, sum all live in this frame  
3     sum  
4 }                        // frame popped here. sum is gone.
```

The heap

- **Layout.** A big pool of memory the allocator hands out chunks of, on request.
- **Allocation.** Your program says "give me 24 bytes" at runtime. The allocator finds a free chunk and returns its address.
- **Deallocation.** Someone has to give it back. In C, you call `free`. In Java or Python, the garbage collector does it. In Rust, *ownership* does it: the compiler inserts the deallocation automatically when the owner goes out of scope.
- **Pays for flexibility.** Sizes can be runtime-determined. Lifetimes can outlive the function that created them.

Stack vs heap, the picture



`v` itself (3 words: pointer, length, capacity) sits on the stack.

The actual numbers `[1, 2, 3, 4, 5]` sit on the heap.

The stack-side `v` holds a *pointer* to the heap buffer.

Rust's defaults

- **Default is the stack.** `let x = 5;` puts `x` in the current frame. No heap involved.
- **Heap is opt-in.** You allocate on the heap by reaching for a type that wraps a heap allocation: `Box<T>`, `Vec<T>`, `String`, `Rc<T>`, and friends. The type system makes the choice visible.
- **There is no `new` keyword.** There is no hidden allocation. If you don't see `Box`, `Vec`, `String`, or a constructor that returns one, no allocation happened.

This is unusual. In Java every object is on the heap; in Python every value is on the heap. In Rust, you pay for the heap only when you ask for it.

LIVE CODING CHALLENGE

Make one `i32` on the stack and one `Box<i32>` on the heap. Print the address of each with `{:p}`, including the address the box *points at*.

Two of those three are close together. Predict which before you look.

Pause here. Try it before the next slide.

Where do these live?

```
1 fn main() {  
2     let on_stack: i32 = 5;  
3     let on_heap: Box<i32> = Box::new(5);  
4  
5     println!("stack {:p}", &on_stack);  
6     println!("box   {:p}", &on_heap);  
7     println!("heap  {:p}", &*on_heap);  
8 }
```

Where do these live?

```
1 fn main() {  
2     let on_stack: i32 = 5;  
3     let on_heap: Box<i32> =  
4  
5     println!("stack {:p}", &on_stack);  
6     println!("box   {:p}", &on_heap);  
7     println!("heap  {:p}", &*on_heap);  
8 }
```

Four bytes in this function's stack frame. That is the whole story

Where do these live?

```
1 fn main() {  
2     let on_stack: i32 = 5;  
3     let on_heap: Box<i32> = Box::new(5);  
4  
5     println!("stack {:p}", &on_stack);  
6     println!("box   {:p}", &on_heap);  
7     println!("heap  {:p}", &*on_heap);  
8 }
```

Two places. The pointer is on the stack; the 5 it points at is on the heap

Where do these live?

```
1 fn main() {  
2     let on_stack: i32 = 5;  
3     let on_heap: Box<i32> = Box::new(5);  
4  
5     println!("stack {:p}", &on_stack);  
6     println!("box   {:p}", &on_heap);  
7     println!("heap  {:p}", &*on_heap);  
8 }
```

Where the *pointer* is: a stack slot
next to **on_stack**

Where do these live?

```
1 fn main() {  
2     let on_stack: i32 = 5;  
3     let on_heap: Box<i32> = Box::new(5);  
4  
5     println!("stack {:p}", &on_stack);  
6     println!("box   {:p}", &on_heap);  
7     println!("heap  {:p}", &*on_heap);  
8 }
```

&* goes through the box first, so this is the *heap* address

Where do these live?

```
1 fn main() {  
2     let on_stack: i32 = 5;  
3     let on_heap: Box<i32> = Box::new(5);  
4  
5     println!("stack {:p}", &on_stack);  
6     println!("box   {:p}", &on_heap);  
7     println!("heap  {:p}", &*on_heap);  
8 }
```

The first two addresses land within a few bytes of each other: both are slots in this function's stack frame. The third is in a completely different region. That is the heap.

2. References and fat pointers

What is a pointer, really?

A reference is a pointer with rules

A reference, `&T`, is an address. The same kind of address a C pointer holds.

What makes it a *reference* and not a raw pointer is what the compiler tracks about it:

- It must point to a valid `T`.
- It cannot outlive the `T` it points to.
- It cannot break the borrow rules (more on this in part 5).

For now, treat `&T` as "an address the compiler is watching."

Two reference flavors

```
1 let mut n = 10;
2 let r: &i32 = &n;           // shared reference: read-only
3 println!("{r}");
4
5 let mut k = 20;
6 let m: &mut i32 = &mut k;   // exclusive reference: read/write
7 *m += 1;
```

- **&T**. Shared. You can have many of these to the same value at once. Read-only.
- **&mut T**. Exclusive. While one exists, no other reference (shared or mutable) to the same value may exist.

Note that **r** and **m** borrow *different* variables. Point them both at **n** and this stops compiling, which is exactly the rule we get to in part 5.

Thin vs fat pointers

A **thin pointer** is one machine word: just an address.

A **fat pointer** is *two* machine words: an address, plus extra metadata the type system needs.

The two cases you meet first:

- **Slices**, `&[T]`. Pointer + **length**. The compiler needs the length to bounds-check indexing.
- **String slices**, `&str`. Same shape: pointer + length, where the bytes are guaranteed UTF-8.

(There's also `&dyn Trait`, which is pointer + vtable. Same fat-pointer idea, different metadata. We'll meet it in the traits lecture.)

Slices, the picture

```
v: Vec<i32>                                [1, 2, 3, 4, 5]  <-- heap
+-----+                                ^  ^
| ptr  ----|-----+                    |
| len: 5    |                    |
| cap: 5    |                    |
+-----+                    |
                                |
s: &[i32]   (a slice of v[1..4])          |
+-----+                                |
| ptr  ----|-----+                    |
| len: 3    |                    |
+-----+                    |
```

A slice is a *view*: a pointer into an existing buffer, plus how many elements the view spans.

LIVE CODING CHALLENGE

Print `size_of` for `&i32`, `&[i32]`, `&str`, `Box<i32>`, and `Box<[i32]>`.

Which of these are 8 bytes, and which are 16?

Pause here. Try it before the next slide.

Pointer sizes

```
1 use std::mem::size_of;
2
3 fn main() {
4     println!("&i32      = {} bytes", size_of::<&i32>());
5     println!("&[i32]    = {} bytes", size_of::<&[i32]>());
6     println!("&str      = {} bytes", size_of::<&str>());
7     println!("Box<i32>    = {} bytes", size_of::<Box<i32>>());
8     println!("Box<[i32]> = {} bytes", size_of::<Box<[i32]>>());
9 }
```

On a 64-bit machine: 8, 16, 16, 8, 16.

The "fat" types carry their length with the pointer. `Box<[i32]>` is fat for the same reason `&[i32]` is.

3. `Box<T>`, your first smart pointer

Heap allocation with an owner

What `Box<T>` is

`Box<T>` is the simplest smart pointer in Rust:

- It **owns** a value on the heap.
- It is a **single owner**. There is no sharing.
- When the `Box` goes out of scope, the value on the heap is dropped automatically. No `free`.

```
1 let b: Box<i32> = Box::new(42);
2 println!("{}", *b);    // deref to read the value
3 // b is dropped here; the heap allocation goes with it.
```

The "smart" in *smart pointer* is the destructor. The pointer knows how to clean up after itself.

When you actually reach for `Box`

Three honest reasons to allocate with `Box`:

1. **Recursive types.** A type that contains itself has no compile-time size unless you put indirection somewhere. `Box` is that indirection. (We're about to do this with a list.)
2. **Trait objects.** `Box<dyn Trait>` lets you hold a value whose concrete type is decided at runtime.
3. **Large values you don't want to move on the stack.** Rare, but real: a 1 MB struct probably shouldn't live in a function frame.

Most of the time, you don't reach for `Box` directly. `Vec`, `String`, `Rc`, and friends already wrap their own heap allocations.

LIVE CODING CHALLENGE

Put a `5` on the heap with `Box::new`. Read it back out with `*`, add one to it, and print both.

Where does the heap allocation get freed, and who writes that line?

Pause here. Try it before the next slide.

Build a `Box`, deref it, drop it

```
1 fn main() {
2     let b = Box::new(5);
3     println!("b points to {}", *b);
4
5     let same: i32 = *b + 1;    // read through the box
6     println!("same = {}", same);
7
8 }    // <-- the box is dropped here. The heap allocation is freed.
```

There's no `free`. There's no garbage collector. The compiler inserts the cleanup at the closing brace.

This is **RAII**: resource acquisition is initialization. When the owner is created the resource is acquired; when the owner is destroyed the resource is released.

4. The canonical example: a list

Why the book uses this

Recursive types need indirection

Suppose you want a singly linked list. The natural definition is:

```
1 enum List {  
2     Cons(i32, List),  
3     Nil,  
4 }
```

Try to compile this. The error:

```
error[E0072]: recursive type `List` has infinite size  
help: insert some indirection (e.g., a `Box`, `Rc`, or `&`)  
      to break the cycle
```

The compiler needs to know how big a `List` is, and a `List` contains a `List`, which contains a `List`, which contains a `List`... no fixed size.

Recursive types need indirection

Suppose you want a singly linked list. The natural definition is:

```
1 enum List {  
2     Cons(i32, List),  
3     Nil,  
4 }
```

A **List** contains a **List**. How big is a **List**?

Try to compile this. The error:

```
error[E0072]: recursive type `List` has infinite size  
help: insert some indirection (e.g., a `Box`, `Rc`, or `&`) to break the cycle
```

The compiler needs to know how big a **List** is, and a **List** contains a **List**, which contains a **List**, which contains a **List**... no fixed size.

The fix: `Box<List>`

A `Box<T>` is one pointer-sized word whenever `T` has a size the compiler knows, no matter how big that size is. Indirection breaks the size recursion.

(`Box<[T]>` and `Box<dyn Trait>` are the exceptions, and they are two words for the same reason `&[T]` is. `List` has a known size once we break the recursion, so `Box<List>` is one word.)

Fixed code

```
1  enum List {
2      Cons(i32, Box<List>),
3      Nil,
4  }
5
6  use List::{Cons, Nil};
7
8  fn main() {
9      let list = Cons(1,
10                     Box::new(Cons(2,
11                                   Box::new(Cons(3,
12                                                 Box::new(Nil))))));
13  }
```

This is the example from *The Rust Programming Language*. It motivates `Box` better than any contrived "put a number on the heap" demo.

LIVE CODING CHALLENGE

Write `fn sum(list: &List) -> i32` that adds up every value in the cons-list.

`match` on the two variants. What does the `Nil` arm return?

Pause here. Try it before the next slide.

Walking the list

```
1 fn sum(list: &List) -> i32 {
2     match list {
3         Cons(v, rest) => v + sum(rest),
4         Nil => 0,
5     }
6 }
7
8 fn main() {
9     let list = Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))));
10    println!("sum = {}", sum(&list));
11 }
```

Notice the signature: `sum` takes `&List`, a *borrow*. It doesn't own the list, doesn't free it, doesn't move it. We're about to make that mechanism the whole point.

But: you wouldn't write a list this way

The cons-list is a teaching device. In real Rust code, when you want "a sequence of `i32`s," you reach for:

```
1 let v: Vec<i32> = vec![1, 2, 3, 4, 5];
```

A `Vec`:

- Stores its elements **contiguously** on the heap (cache-friendly; one allocation, not N).
- Tracks length and capacity, so it can grow.
- Hands out slices `&[i32]` for read views and `&mut [i32]` for write views.

The cons-list teaches you about `Box`. The `Vec` is what you use. We need the `Vec` for the next part.

Graphs and doubly-linked lists

What if you want a doubly-linked list (each node points to the next *and* the previous)? Or a graph (nodes with many neighbors, possibly cyclic)?

In Rust you have three honest options:

- `Rc<RefCell<Node>>`. Shared ownership with interior mutability. Works, but heavy. We'll meet `Rc` at the end of this lecture; `RefCell` later.
- **Indices into a `Vec<Node>`**. Replace pointers with `usize` indices. The `Vec` owns all nodes; edges are just numbers. Often the right answer.
- **`unsafe` with raw pointers**. Available, occasionally necessary, almost never first.

The borrow checker is not banning these structures; it's nudging you toward representations that are easier to reason about. Most of the time, the nudge is correct.

Recap

1. **Two places a value can live.** The stack is fast, ordered, and fixed-size; the heap is flexible and costs an allocator request.
2. **Rust defaults to the stack.** The heap is opt-in and visible in the type: `Box`, `Vec`, `String`, `Rc`. There is no hidden allocation.
3. **A reference is an address the compiler is watching.** `&T` shared, `&mut T` exclusive.
4. **Fat pointers carry metadata.** `&[T]` and `&str` are two words, pointer plus length, so `size_of::<&[i32]>()` is 16.
5. **`Box<T>` is the simplest heap allocation:** one owner, freed when it drops, pointer-sized whatever it holds.
6. **Recursive types need indirection**, which the cons-list example really demonstrates. You would not write that list, though: `Vec` is one allocation instead of N.

Exercises

1. Print the addresses of a stack `i32`, a `Box<i32>`, and the value the box points at. Explain which two are close and why.
2. Print `size_of` for `&i32`, `&[i32]`, `&str`, `Box<i32>`, and `Box<[i32]>`. Explain each number in one sentence.
3. Write the cons-list without `Box` and read `error[E0072]` carefully. What exactly cannot the compiler compute?
4. Write a recursive `sum` over the cons-list, then write the same thing over a `Vec<i32>`. Compare the two for length and for how much you had to think.
5. Draw the memory diagram for `let v = vec![1, 2, 3];` showing what is on the stack and what is on the heap, with the three words of the `Vec` labelled.

Reading

- **The Rust Programming Language**, chapter 15 (Smart Pointers), especially 15.1 on `Box`.
- **Rust by Example**, the sections on primitives and pointers.
- **Course supplements.** [stack-and-heap-primer.md](#) if this was your first exposure to the stack and heap; [coming-from-c.md](#) if you arrived with C or C++.

Next: Lecture 5

Ownership and Borrowing

You know where values live. Now: who is allowed to touch them.